

# Phép toán Tensor: Element-wise và Broadcasting

## Phép toán theo phần tử (Element-wise):

- Áp dụng độc lập cho từng mục trong tensor (Cộng, Trừ, Hàm ReLU...).
- Được thực thi song song qua BLAS siêu tốc trên kiến trúc GPU.

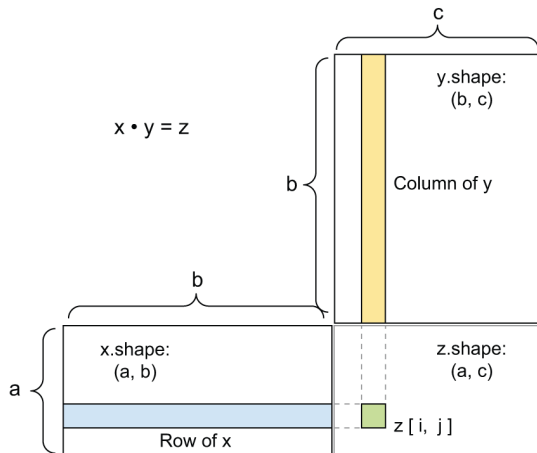
## Phát sóng (Broadcasting):

- Cho phép tính toán giữa 2 tensor khác kích thước (VD: Ma trận + Vector).
- Tensor nhỏ được "lặp lại" dọc theo trục mới để khớp hình dạng.

```
1 import numpy as np
2 X = np.random.random((32, 10))
3 y = np.random.random((10,))
4 Z = X + y # y tu dong phat song thanh (32, 10) roi moi cong.
5
```

# Sản phẩm Tensor (Tích vô hướng - Dot Product)

- Không giống phép nhân từng phần tử, **Tích Tensor** (`matmul` hoặc toán tử `@`) là phép toán phổ biến nhất.
- Tích ma trận:** Kết hợp các hàng ma trận 1 với cột ma trận 2.
- Khả năng tương thích hình dạng:  
 $(a, b) \cdot (b, c) \rightarrow (a, c)$



Hình 2.5: Sơ đồ hộp minh họa tích ma trận

# Định hình lại Tensor (Reshaping & Transposing)

- Định hình lại (Reshape) sắp xếp lại các hàng và cột để đạt được hình dạng mục tiêu mà không thay đổi tổng số phần tử.

```
1 x = np.array([[0., 1.],  
2             [2., 3.],  
3             [4., 5.]]) # Shape: (3, 2)  
4 x = x.reshape((6, 1)) # Tro thành cot 6 phan tu  
5
```

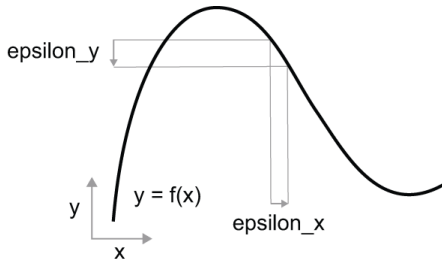
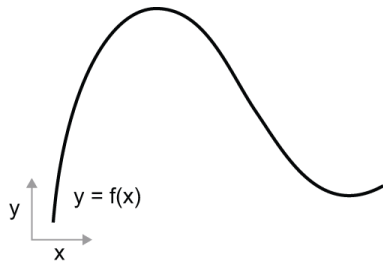
- Chuyển vị (Transpose): Hoán đổi hàng thành cột.

```
1 x = np.zeros((300, 20))  
2 x = np.transpose(x) # Shape tro thanh (20, 300)  
3
```

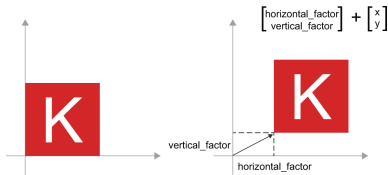
# Ý nghĩa Hình học (1): Hàm số và Tính liên tục

Tất cả phép toán tensor đều có ý nghĩa biến đổi hình học.

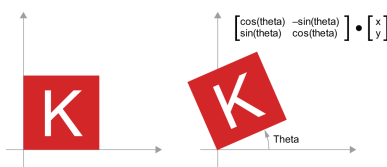
- Một mạng nơ-ron thực chất là một **hàm toán học** khổng lồ.
- Hàm số này phải **liên tục** và **trơn**.



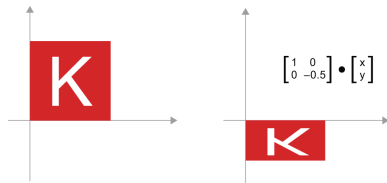
# Ý nghĩa Hình học (2): Tịnh tiến, Phóng to, Quay



Hình 2.7: **Tịnh tiến:** Dời hình bằng phép cộng vector



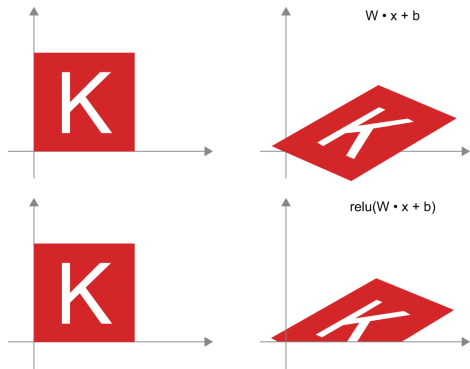
Hình 2.8: **Quay:** Tích ma trận  $2 \times 2$  (Lượng giác)



Hình 2.9: **Thu phóng:** Tích ma trận đường chéo

## Ý nghĩa Hình học (3): Phép biến đổi Affine & Dense layer

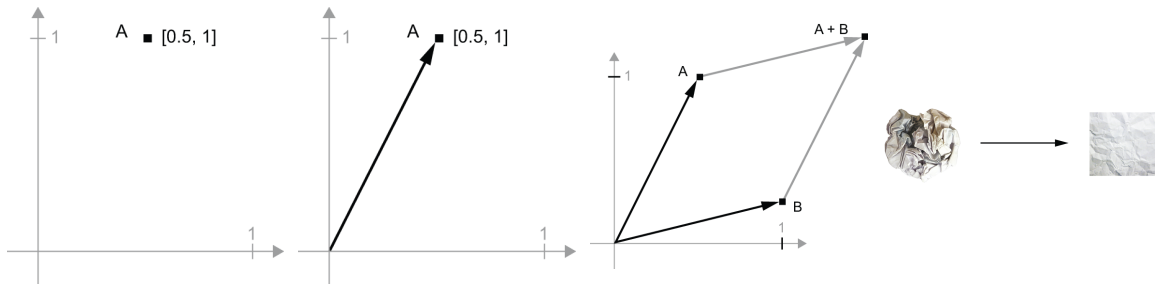
- **Phép biến đổi Affine** là sự kết hợp của biến đổi tuyến tính (nhân  $W$ ) và tịnh tiến (cộng  $b$ ).
- Lớp Dense:  $y = W \cdot x + b$ .
- **Kích hoạt phi tuyến (ReLU)**: Giúp bóp méo không gian affine, tạo ra các ranh giới phức tạp (để phân tách dữ liệu).



Hình 2.10: Biến đổi Affine & theo sau là ReLU

## Ý nghĩa Hình học (4): Gỡ rối dữ liệu

Học sâu bản chất là chuỗi biến đổi hình học tinh vi để "gỡ rối" (untangle) các đa tạp dữ liệu phức tạp đan xen nhau.



**Hình 2.11:** Quá trình phân tách dữ liệu qua các lớp ẩn giống như việc từ từ vuốt phẳng một quả bóng giấy vò nát.

# Vòng lặp Đào tạo (Training Loop)

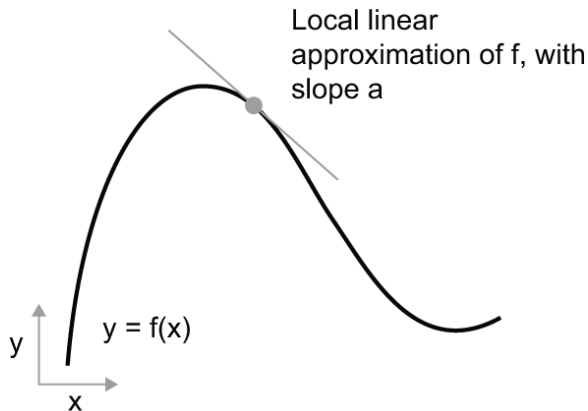
- Khởi tạo ngẫu nhiên trọng số mô hình  $W$  và  $b$ .
- Lặp lại các bước sau (Vòng lặp đào tạo):
  - ① Lấy một lô ngẫu nhiên dữ liệu huấn luyện  $x$  và nhãn  $y_{\text{true}}$ .
  - ② **Forward Pass:** Chạy mô hình trên  $x$  để dự đoán  $y_{\text{pred}}$ .
  - ③ Tính hàm Mất mát (Loss) để đo lường sai số.
  - ④ Cập nhật tất cả các trọng số theo hướng giảm Loss.
- Làm sao để biết thay đổi trọng số bao nhiêu và hướng nào? Phương pháp: **Gradient Descent**.



# Khái niệm Đạo hàm (Derivative)

Đạo hàm  $f'(x)$  thể hiện **độ dốc (slope)** của tiếp tuyến với hàm số  $f(x)$  tại điểm  $x$ .

- Nếu  $f'(x) > 0$ , tăng  $x$  sẽ làm hàm tăng.
- Nếu  $f'(x) < 0$ , tăng  $x$  sẽ làm hàm giảm.
- $\Rightarrow$  Muốn giảm giá trị  $f(x)$  (Loss), chỉ cần đi **ngược hướng** với đạo hàm.

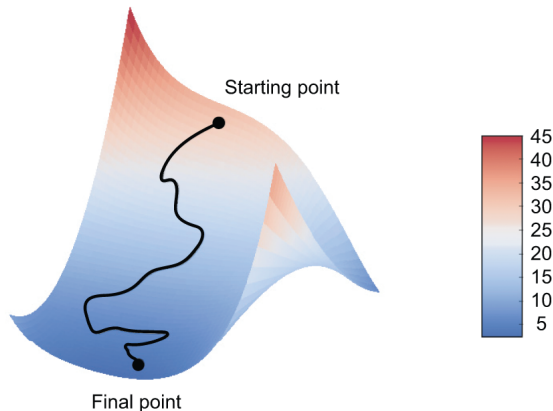


Hình 2.12: Đạo hàm là hệ số góc của tiếp tuyến

# Đạo hàm của Tensor: Gradient

Sự tổng quát hóa của đạo hàm trong không gian đa chiều (nhiều tham số trọng số) gọi là **Gradient**.

- Gradient biểu thị **độ cong** của bề mặt hàm Loss đa chiều.
- Vectơ Gradient luôn chỉ về hướng hàm  $L$  tăng nhanh nhất.
- **Ngược hướng Gradient** là con đường ngắn nhất xuống thung lũng Loss.

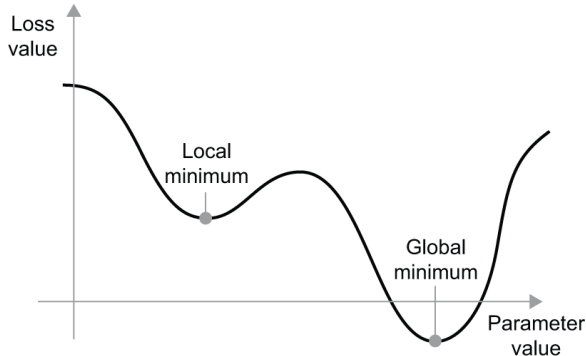


Hình 2.13: Mặt cong 3D của Loss function

# Cực tiểu Toàn cục và Cực tiểu Cục bộ

Mục tiêu của học sâu là tìm **Cực tiểu toàn cục (Global Minimum)**, nơi sai số đạt mức thấp nhất.

- Tại cực tiểu, Gradient bằng 0 ( $\nabla L = 0$ ).
- Với bề mặt Loss phức tạp, mô hình rất dễ mắc kẹt tại **Cực tiểu cục bộ (Local Minimum)**.



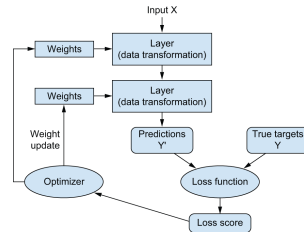
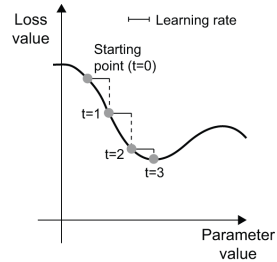
Hình 2.14: Local Minimum vs Global Minimum

# Thuật toán SGD và Momentum

**Stochastic Gradient Descent (SGD):** Cập nhật dựa trên một mẻ nhỏ ngẫu nhiên.

$$W \leftarrow W - \text{learning\_rate} \times \nabla L$$

- **Động lượng (Momentum):** Lấy cảm hứng từ một quả bóng lăn dốc, sử dụng vận tốc quá khứ để giúp quả bóng thoát khỏi các cực tiểu cục bộ hẹp.

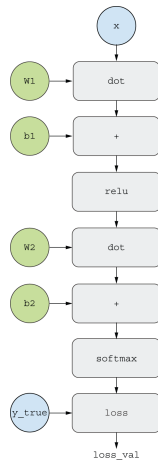


Hình 2.15: Học Gradient ngẫu nhiên & Vòng lặp

# Đồ thị Tính toán (Computation Graph)

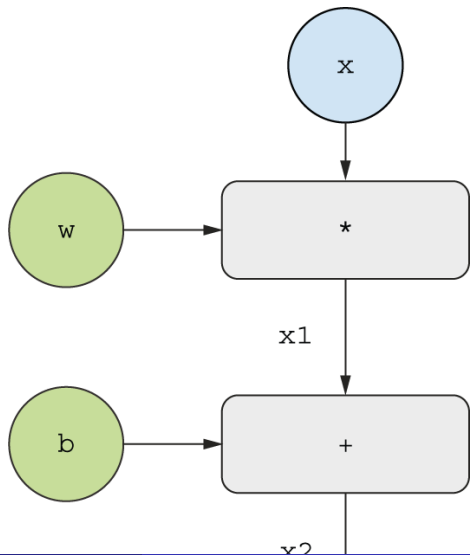
Làm sao để tính đạo hàm của hàng triệu trọng số? Sử dụng **Lan truyền ngược**.

- Cốt lõi của nó là **Đồ thị tính toán**.
- Các phép toán Tensor trở thành các "nút" (node), dữ liệu trở thành các cạnh.
- Biến tính toán toán học thành dữ liệu cấu trúc phục vụ máy tính.



Hình 2.16: Biểu diễn đồ thị cho mô hình mạng nơ-ron

# Ví dụ lan truyền thuận (Forward Pass)



# Lan truyền ngược (Backpropagation) & Quy tắc Chuỗi

Bằng cách áp dụng **Quy tắc chuỗi (Chain Rule)**:

$$\frac{d(f(g(x)))}{dx} = f'(g(x)) \cdot g'(x)$$

- Bắt đầu từ Loss, ta lan truyền đạo hàm

